Arboles Generales

Implementación en JAVA
Ejemplos de parciales

Arboles Generales

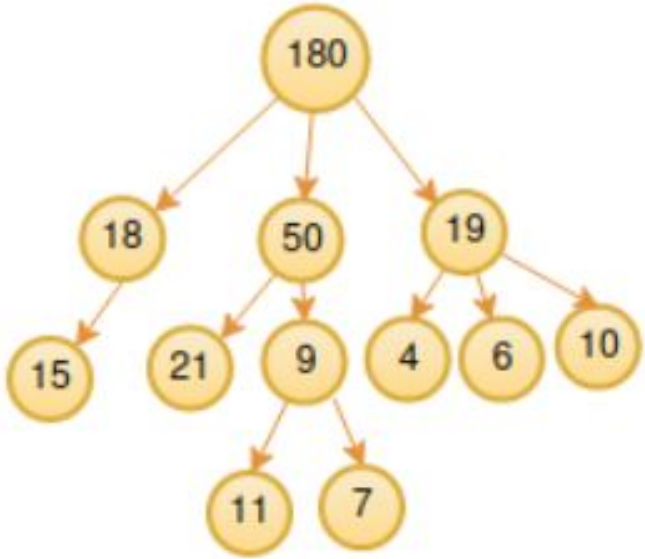
Estructura

GeneralTree<T>

tp3

- data: T
- children: List<GeneralTree<T>>

- GeneralTree(data: T): void
- GeneralTree(data: T, children: List<GeneralTree<T>>): void
- getData(): T
- setData(data: T): void
- setChildren(children: List<GeneralTree<T>>): void
- getChildren(): List<GeneralTree<T>>
- addChild(child: GeneralTree<T>): void
- isLeaf(): boolean
- hasChildren(): boolean
- isEmpty(): boolean
- removeChild(child: GeneralTree<T>): void
- printPreOrder(): void
- preOrder(): List<T>
- preOrder(l: List<T>): void
- postOrder(): void
- traversalLevel(tree: GeneralTree<T>): List<T>



Arboles Generales

Código Fuente – Constructores, this()

```
package tp3;

public class GeneralTree<T> {
 private T data;
 private List<GeneralTree<T>> children =
                       new LinkedList<GeneralTree<T>>();

 public GeneralTree(T data) {
      this.data = data;
 }
 public GeneralTree(T data,List<GeneralTree<T>> children){
      this(data);
     this.children = children;
 }

 public boolean hasChildren() {
  return children!=null && !children.isEmpty();
 }

 public void setChildren(List<GeneralTree<T>> children) {
   if (children != null)
       this.children = children;
 }

 public List<GeneralTree<T>> getChildren() {
   return this.children;
 }
```

```
 public void addChild(GeneralTree<T> child) {
   getChildren().add(child);
 }

 public T getData() {
   return data;
 }

 public void setData(T data) {
   this.data = data;
 }

 public boolean isLeaf() {
  return !hasChildren();
 }

 public boolean isEmpty() {
    return data==null && !this.hasChildren();
 }

 public void removeChild(GeneralTree<T> child) {
   if (this.hasChildren()) {
       children.remove(child);
   }
}
 . . .
}
```

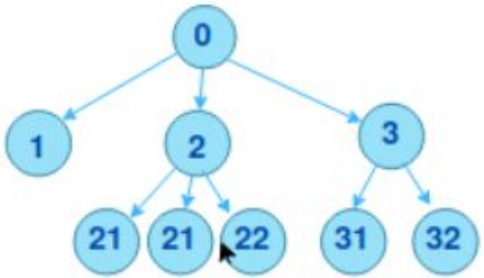
Arboles Generales
Recorrido PreOrden

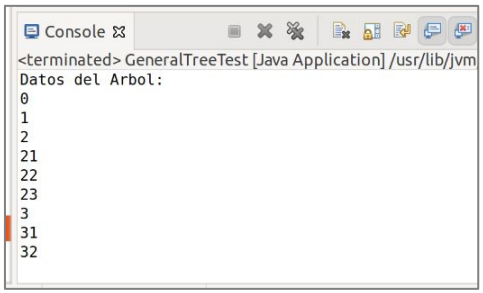
Implementar un método en **GeneralTree** que imprima en preorder los elementos del árbol.

```java
package tp3;
public class GeneralTree<T> {
  ...
 private void preOrder() {
   System.out.println(getData());
   List<GeneralTree<T>> children = this.getChildren();
   for (GeneralTree<T> child: children) {
       child.preOrder();
   }
 }
}
```

```java
package tp3;
public class GeneralTree<T> {
  ...
  private void preOrder() {
    System.out.println(getData());
    List<GeneralTree<T>> children = this.getChildren();
    Iterator<GeneralTree<T>> it = children.iterator();
    while (it.hasNext()) {
        GeneralTree<T> child = it.next();
        child.preOrder();
    }
  }
}
```

```java
public class GeneralTreeTest {
  public static void main(String[] args) {
      GeneralTree<String> a1 = new GeneralTree<String>("1");
      List<GeneralTree<String>> children2 = new LinkedList<GeneralTree<String>>();
      children2.add(new GeneralTree<String>("21"));
      children2.add(new GeneralTree<String>("22"));
      children2.add(new GeneralTree<String>("23"));
      GeneralTree<String> a2 = new GeneralTree<String>("2", children2);
      List<GeneralTree<String>> children3 = new LinkedList<GeneralTree<String>>();
      children3.add(new GeneralTree<String>("31"));
      children3.add(new GeneralTree<String>("32"));
      GeneralTree<String> a3 = new GeneralTree<String>("3", children3);
      List<GeneralTree<String>> childen = new LinkedList<GeneralTree<String>>();
      childen.add(a1);childen.add(a2);childen.add(a3);
      GeneralTree<String> a = new GeneralTree<String>("0", childen);
      System.out.println("Datos del Arbol: ");
      a.printPreOrder();
  }
}
```

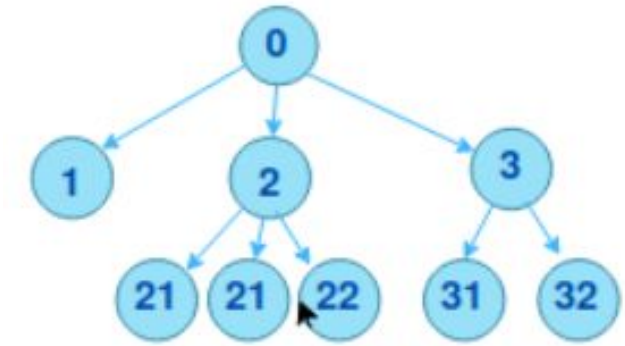




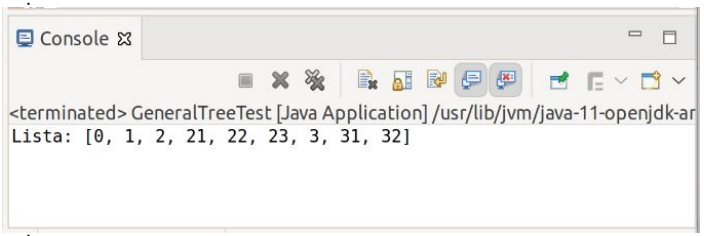
Arboles Generales
Recorrido Preorden

Implementar un método en **`GeneralTree`** que retorne una lista con los datos del árbol recorrido en preorden

```
package tp3;
public class GeneralTree<T> {
   ...
  public List<T> preOrder() {
      List<T> lis = new LinkedList<T>();
      this.preOrder(lis);
      return lis;
  }
  private void preOrder(List<T> l) {
      l.add(this.getData());
      List<GeneralTree<T>> children = this.getChildren();
      for (GeneralTree<T> child: children) {
            child.preOrder(l);
      }
  }
}
```




```
public class GeneralTreeTest {
  public static void main(String[] args) {
      GeneralTree<String> a1 = new GeneralTree<String>("1");
      List<GeneralTree<String>> children2 = new LinkedList<GeneralTree<String>>();
      children2.add(new GeneralTree<String>("21"));
      children2.add(new GeneralTree<String>("22"));
      children2.add(new GeneralTree<String>("23"));
      GeneralTree<String> a2 = new GeneralTree<String>("2", children2);
      . . .
      List<GeneralTree<String>> childen = new LinkedList<GeneralTree<String>>();
      childen.add(a1);childen.add(a2);childen.add(a3);
      GeneralTree<String> a = new GeneralTree<String>("0", childen);

      System.out.println("Lista: "+a.preOrder());
  }
}
```



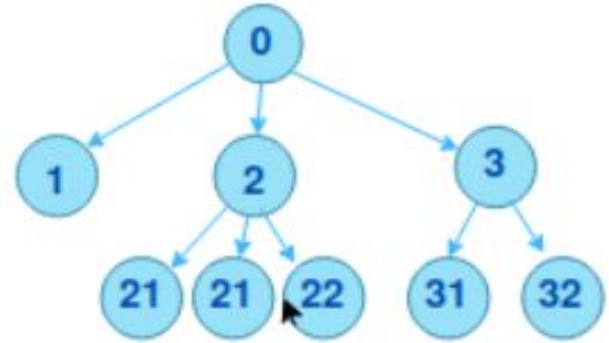
Arboles Generales

Recorrido por niveles

Recorrido por niveles en una clase externa a `ArbolGeneral`. El método retorna una lista con los valores del árbol.

```
public void porNiveles() {
   encolar(raíz);
   mientras cola no se vacíe {
      v ← desencolar();
      imprimir (dato de v);
      para cada hijo de v
                encolar(hijo);
   }
}
```

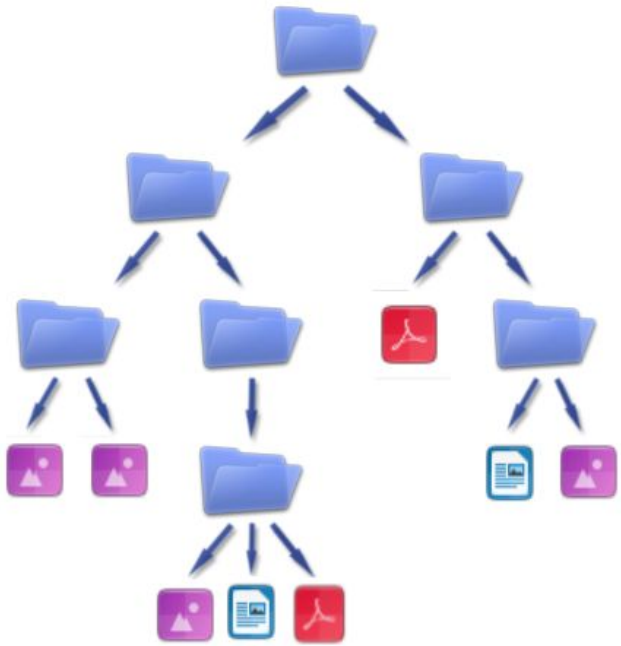
```
public List<T> traversalLevel(GeneralTree<T> tree) {

     List<T> result = new LinkedList<T>();
     GeneralTree<T> tree_aux;

     Queue<GeneralTree<T>> queue = new Queue<GeneralTree<T>>();
     queue.enqueue(tree);
     while (!queue.isEmpty()) {
           tree_aux = queue.dequeue();
           result.add(tree_aux.getData());
           List<GeneralTree<T>> children = tree_aux.getChildren();
           for (GeneralTree<T> child: children) {
                queue.enqueue(child);
           }
      }
      return result;
    }
```



Arboles Generales

Ejercicio: Devolver las imágenes de un nivel

Dado un árbol que representa una estructura de directorios como la que muestra la imagen, implementar un método que reciba un nivel y retorne una lista con las imágenes encontradas en ese nivel.
Modelar el recurso para representar las carpetas y los archivos.

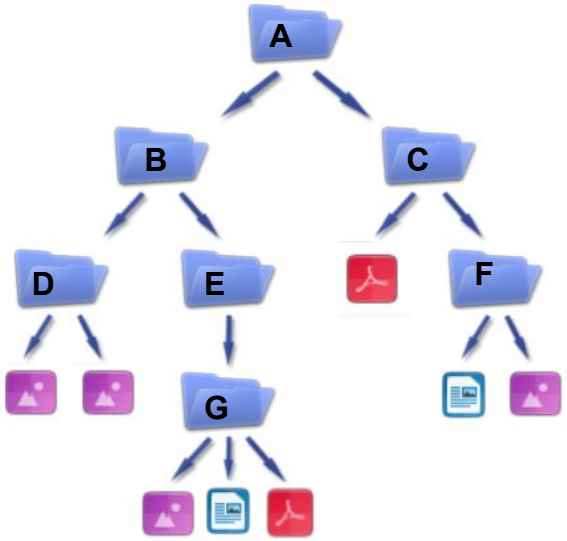


```
public class Recurso {
  String nombre;
  String tipo; // archivo o carpeta


  public Recurso(String nombre, String tipo) {
       this.nombre = nombre;
       this.tipo = tipo;
  }

 public boolean esImagen() {
   if (tipo.equals("archivo")) {
      String ext = nombre.substring(nombre.indexOf('.') + 1);
      if (ext.equals("jpg") || ext.equals("png") || ext.equals("jpeg"))
             return true;
   }
   return false;
  }
  . . .
}
```

Arboles Generales

Ejercicio de parcial – Devolver imágenes

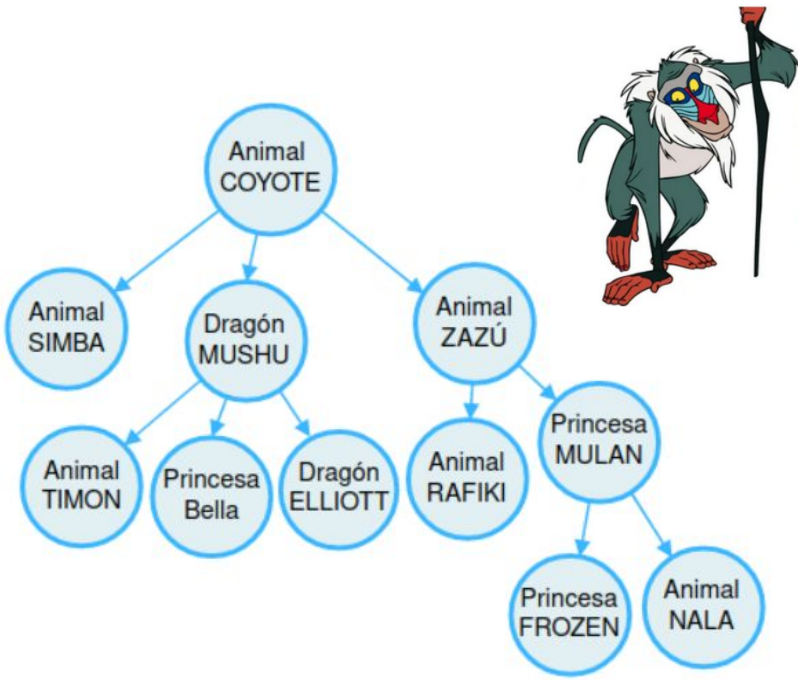
```
public class DevolverImagenes {
  public static List<Recurso> getImagenes(GeneralTree<Recurso> ag, int nivel_pedido) {
      List<Recurso> result = new LinkedList<Recurso>();
      Queue<GeneralTree<Recurso>> cola = new Queue<GeneralTree<Recurso>>();
      GeneralTree<Recurso> arbol_aux;
      cola.enqueue(ag);
      cola.enqueue(null);
      int nivel = 0;
      while (!cola.isEmpty() && nivel <= nivel_pedido) {
            arbol_aux = cola.dequeue();
            if (arbol_aux != null) {
                  Recurso rec = arbol_aux.getData();
                  if (nivel == nivel_pedido && rec.esImagen())
                        result.add(arbol_aux.getData());
                  if (arbol_aux.hasChildren() && nivel < nivel_pedido ) {
                        List<GeneralTree<Recurso>> children = arbol_aux.getChildren();
                        for (GeneralTree<Recurso> child: children) {
                              cola.enqueue(child);
                        }
                  }
            } else {
                  if (!cola.isEmpty()) {
                        nivel++;
                        cola.enqueue(null);
                  }

            }
      }
      return result;
    }
 }
```



Arboles Generales

Ejercicio de parcial – Encontrar a la Princesa

Dado un árbol general compuesto por personajes, donde puede haber dragones, princesas y otros, se denominan nodos accesibles a aquellos nodos tales que a lo largo del camino del nodo raíz del árbol hasta el nodo (ambos inclusive) no se encuentra ningún dragón.

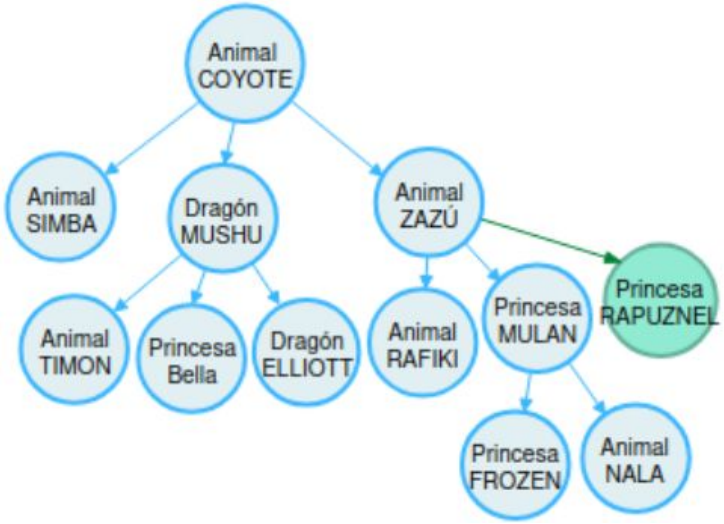


Implementar un método que devuelva una lista con un camino desde la raíz a una Princesa sin pasar por un Dragón –sin necesidad de ser el más cercano a la raíz-. Asuma que existe al menos un camino accesible.

Arboles Generales

Ejercicio de parcial – Encontrar a la Princesa

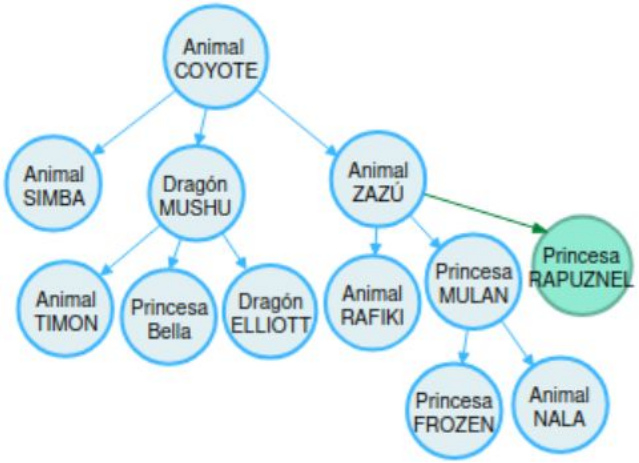
```
package parcial.juego;

public class Personaje {
 private String nombre;
 private String tipo;    //Dragon, Princesa, Animal, etc.

 public Personaje(String nombre, String tipo) {
  this.nombre = nombre;
  this.tipo = tipo;
 }
 public String getNombre() {
  return nombre;
 }
 public void setNombre(String nombre) {
  this.nombre = nombre;
 }
  . . .

 public boolean esDragon(){
  return this.getTipo().equals("Dragon");
 }
 public boolean esPrincesa(){
  return this.getTipo().equals("Princesa");
 }
}
```



Arboles Generales

Ejercicio de parcial – Encontrar a la Princesa
Versión I con for

```
public class JuegoPersonaje_v1 {
   public static List<Personaje> encontrarPrincesa(GeneralTree<Personaje> arbol) {
      List<Personaje> lista = new LinkedList<Personaje>();
      lista.add(arbol.getData());
      List<Personaje> camino = new LinkedList<Personaje>();
      encontrarPrincesa(arbol, lista, camino);
      return camino;
   }

   private static void encontrarPrincesa(GeneralTree<Personaje> arbol, List<Personaje> lista,
                                                                        List<Personaje> camino) {
    Personaje p = arbol.getData();
    if (p.esPrincesa()) {
       camino.addAll(lista);
    }
    if (camino.isEmpty()) {
       List<GeneralTree<Personaje>> children = arbol.getChildren();
       for (GeneralTree<Personaje> child: children) {
            if (!child.getData().esDragon()) {
                 lista.add(child.getData());
                 encontrarPrincesa(child, lista, camino);
                 if (lista.get(lista.size()-1).esPrincesa()) break;
                 lista.remove(lista.size() - 1);
            }
       }
    }
   }
}
```

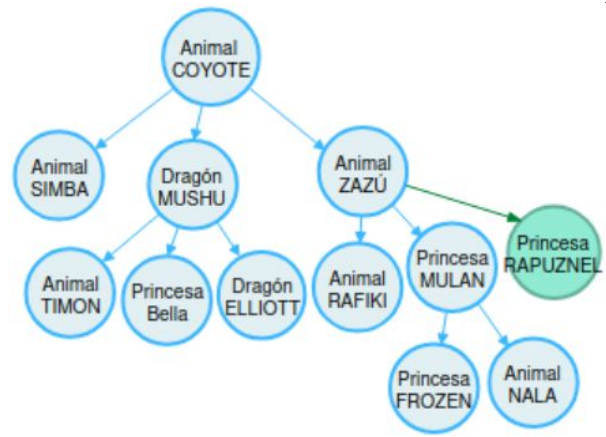


- Notar que si no se pone el break, sigue procesando
- Como al encontrar un camino hay que terminar, mejor con while

Arboles Generales

Ejercicio de parcial – Encontrar a la Princesa
Versión I con while

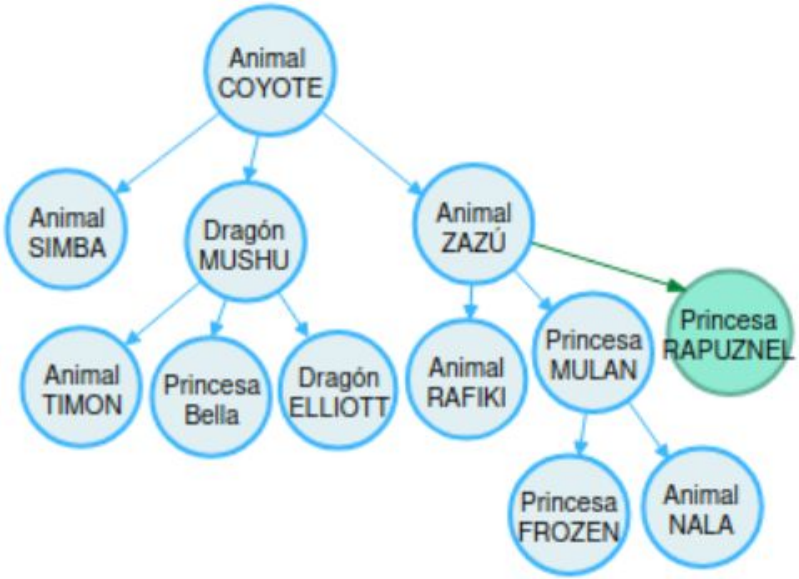
```
public class JuegoPersonaje_v1 {

   public static List<Personaje> encontrarPrincesa(GeneralTree<Personaje> arbol) {
       List<Personaje> lista = new LinkedList<Personaje>();
       lista.add(arbol.getData());
       List<Personaje> camino = new LinkedList<Personaje>();
       encontrarPrincesa(arbol, lista, camino);
       return camino;
   }

   private static void encontrarPrincesa(GeneralTree<Personaje> arbol, List<Personaje> lista,
                                                                       List<Personaje> camino) {
     Personaje p = arbol.getData();
     if (p.esPrincesa()) {
        camino.addAll(lista);
     }
      // if (camino.isEmpty()) {
      Iterator<GeneralTree<Personaje>> it = arbol.getChildren().iterator();
      while (it.hasNext() && camino.isEmpty()) {
            GeneralTree<Personaje> child = it.next();
            if (!child.getData().esDragon()) {
                  lista.add(child.getData());
                  encontrarPrincesa(child, lista, camino);
                  lista.remove(lista.size() - 1);
            }
      }
      // }
   }
}
```



Arboles Generales

Ejercicio de parcial – Encontrar a la Princesa
Versión II

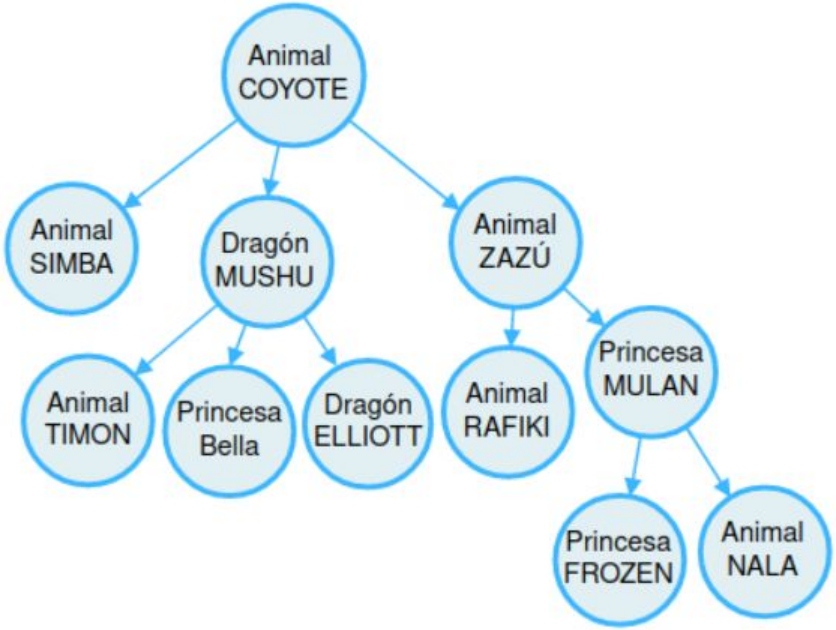
```
public class JuegoPersonaje_v2 {

    public static List<Personaje> encontrarPrincesa(GeneralTree<Personaje> arbol) {
       List<Personaje> lista = null;
       if (arbol.getData().esPrincesa() || arbol.getData().esDragon() || arbol.isLeaf()) {
             if (arbol.getData().esPrincesa()) {
                   Personaje p = arbol.getData();
                   lista=new LinkedList<Personaje>();
                   lista.add(0, p);
             }
             return lista;
       }

       List<GeneralTree<Personaje>> hijos = arbol.getChildren();
       Iterator<GeneralTree<Personaje>> it = hijos.iterator();
       while (it.hasNext() && lista==null) {
             lista = encontrarPrincesa(it.next());
             if (lista!=null) {
                   lista.add(0, arbol.getData());
                   // break; o lista==null en el while
             }
       }
       return lista;
    }
}
```



Arboles Generales

Ejercicio de parcial – Encontrar a la Princesa

```
package tp3.juego;
...
public class JuegoTest {
public static void main(String[] args) {
   Personaje p0 = new Personaje("COYOTE", "Animal");
   Personaje p1 = new Personaje("SIMBA", "Animal");
   Personaje p2 = new Personaje("MUSHU", "Dragon");
   Personaje p3 = new Personaje("TIMON", "Animal");
   Personaje p4 = new Personaje("ELLIOT", "Dragon");
   Personaje p5 = new Personaje("Bella", "Princesa");
   Personaje p6 = new Personaje("ZAZU", "Animal");
   Personaje p7 = new Personaje("RAFIKI", "Animal");
   Personaje p8 = new Personaje("MULAN", "Princesa");
   Personaje p9 = new Personaje("FROZEN", "Princesa");
   Personaje p10 = new Personaje("NALA", "Animal");
   //SIMBA
   GeneralTree<Personaje> a1 = new GeneralTree<Personaje>(p1);
   //MUSHU
   GeneralTree<Personaje> a21 = new GeneralTree<Personaje>(p3);
   GeneralTree<Personaje> a22 = new GeneralTree<Personaje>(p4);
   GeneralTree<Personaje> a23 = new GeneralTree<Personaje>(p5);
   List<GeneralTree<Personaje>> hijosa2 = new LinkedList<GeneralTree<Personaje>>();
   hijosa2.add(a21);
   hijosa2.add(a22);
   hijosa2.add(a23);
   GeneralTree<Personaje> a2 = new GeneralTree<Personaje>(p2, hijosa2);
   . . .
   JuegoPersonaje_v2.encontrarPrincesa(a);
 }
}
```

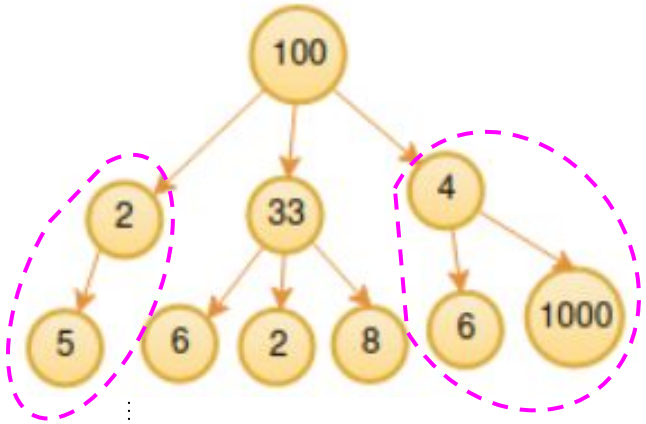


Arboles Generales

Encontrar árboles de padres más pequeño

Implementar una clase con un método que reciba un árbol general de enteros y guarde todos los árboles cuya raíz tenga un valor más pequeño que la suma de los valores de sus hijos.

```
public class BuscarPadreMenor {

 public static int buscar(GeneralTree<Integer> ag) {
   if (ag.isLeaf()) {
      return ag.getData();
   }
   int cont = 0;
   List<GeneralTree<Integer>> children = ag.getChildren();
   for (GeneralTree<Integer> child : children)
      cont = cont + buscar(child);

   if (ag.getData() < cont) {
      Repositorio.agregar(ag);
   }
   return ag.getData();
 }
}
```



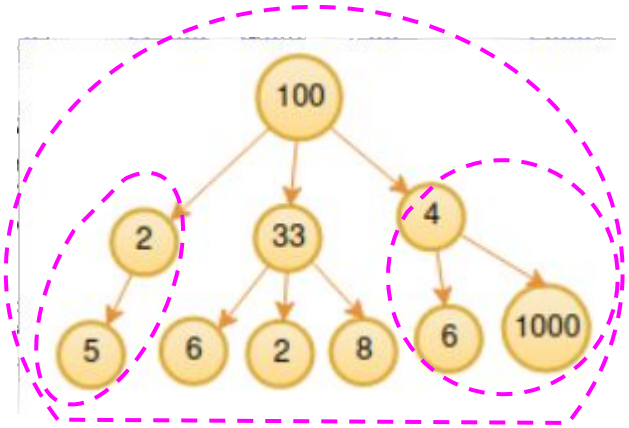

```
public class Repositorio {
 private static List<GeneralTree<Integer>> lista =
                    new LinkedList<GeneralTree<Integer>>();

 public static void agregar(GeneralTree<Integer> a) {
   lista.add(a);
 }
 public static List<GeneralTree<Integer>> devolverLista(){
   return lista;
 }
 public static void limpiartLista() {
  lista = new LinkedList<GeneralTree<Integer>>();}
 }
}
```

Arboles Generales

Encontrar árboles de padres más pequeño

¿Qué cambios se deberían hacer para devolver todos los árboles cuya raíz tenga un valor más pequeño que la suma de los valores de sus descendientes? No se puede usar una clase auxiliar.

```
public class BuscarPadreMenor {

  public static List<GeneralTree<Integer>> buscarHijos(GeneralTree<Integer> ag) {
     List<GeneralTree<Integer>> listaArboles = new LinkedList<GeneralTree<Integer>>();
     buscarHijos(ag, listaArboles);
     return listaArboles;
  }

  private static int buscarHijos(GeneralTree<Integer> ag, List<GeneralTree<Integer>> listaArboles) {
     if (ag.isLeaf()) {
      return ag.getData();
     }
     int cont = 0;
     List<GeneralTree<Integer>> children = ag.getChildren();
          for (GeneralTree<Integer> child : children) {
               cont = cont + buscarHijos(child, listaArboles);
     }
     if (ag.getData()<cont) {
          listaArboles.add(ag);
     }
     return cont+ag.getData();
   }
}
```



Arboles Generales

Sistema de numeración Gematría

Antiguamente el pueblo judío usaba un sistema de numeración llamado Gematria para asignar valores a las letras y así “ocultar” nombres, de aquí que se asocia el nombre de Nerón César al valor 666 (la suma de los valores de sus letras).
Usted cuenta con una estructura como la que aparece en el gráfico, donde **cada camino en este árbol representa un nombre**. Cada nodo **contiene un valor** asociado a una letra, excepto el nodo raíz que contiene el valor 0 y no es parte de ningún nombre, y simplemente significa “comienzo”. **Un nombre completo SIEMPRE es un camino que comienza en la raíz y termina en una hoja.**

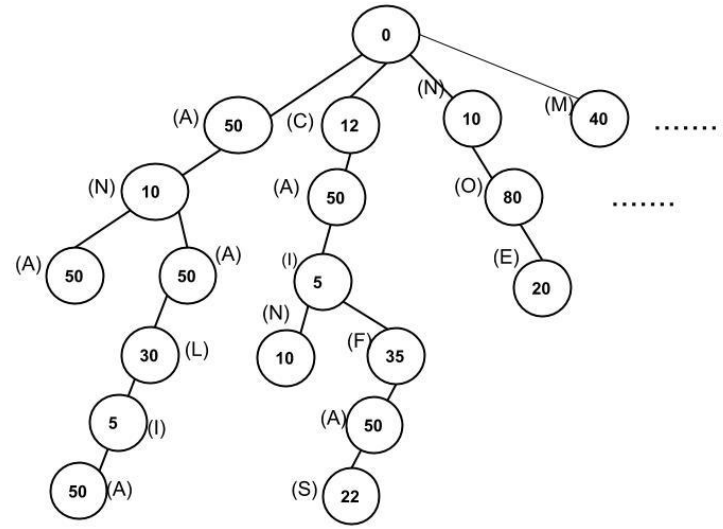
Su tarea será: **dado un valor numérico**, **contar cuantos nombres** completos suman exactamente ese **valor**. Usted recibe el árbol con las letras ya sustituidas por sus valores; las letras ya no importan.

Para esto, escriba una clase llamada **`ProcesadorGematria`** (que NO contenga variables de instancia), con sólo un método público con la siguiente firma:

```
public int contar(xxx, int valor)
```

estructura que contiene los números

valor es el valor que se debería obtener al sumar el valor de las letras de un nombre



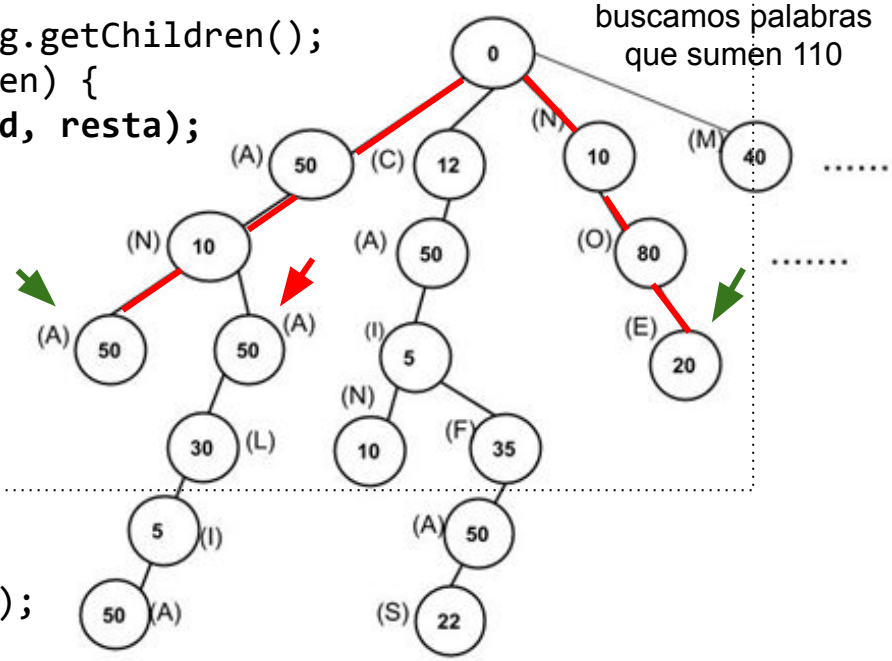
Estructura de números que representa nombres. Dado el valor 110 el método debe devolver 2 (porque ANA y NOE suman 110), dado el valor 77 el método debe devolver 1(porque sólo CAIN suma 77).

Arboles Generales

Sistema de numeración Gematría

```
package tp3.gematria;
import java.util.List;
import tp3.GeneralTree;

public class Gematria {
    public static int contadorGematria(GeneralTree<Integer> ag, int valor) {
      int resta = valor - ag.getData();
      if (ag.isLeaf() && resta == 0)
           return 1;
      else {
           int cont = 0;
           if (resta > 0) {
                List<GeneralTree<Integer>> children = ag.getChildren();
                for (GeneralTree<Integer> child: children) {
                     cont = cont + contadorGematria(child, resta);
                 }
           }
           return cont;
      }
    }
}
```

supongamos que buscamos palabras que sumen 110



Invocación

```
int cant = Gematria.contadorGematria(ag,110);
```